TP 1 : Programmation et transactions

Programmation PL/pgSQL, procédures, triggers et concurrence

Module : Bases de Données Avancées
Niveau : M1
Responsable du TP : M. Alachaher
Année universitaire : 2026/2027

Objectif

Apprendre à programmer dans PostgreSQL avec PL/pgSQL, automatiser des traitements métier avec des fonctions, procédures et triggers, puis garantir la cohérence des données grâce aux transactions. Chaque partie correspond à une séance.

Préparation

Exécuter le fichier `dataset_tp1.sql` dans le *Query Tool* de pgAdmin, dans une base de données de travail **Specilaité_numGroupe**. Le script crée le schéma `tp1_banque` et les tables suivantes :

Table	Rôle et attributs
`client`	`id_client`, `nom`, `prenom`, `ville` et `date_inscription`.
`compte`	`id_compte`, `id_client`, `type_compte`, `solde`, `date_ouverture` et `actif`.
`operation`	`id_operation`, `id_compte`, `type_operation`, `montant`, `date_operation` et `commentaire`.
`historique_solde`	`id_historique`, `id_compte`, `ancien_solde`, `nouveau_solde` et `date_modification`. Cette table est initialement vide.

Le script contient 8 clients, 10 comptes et 10 opérations historiques. Ces opérations sont **déjà intégrées aux soldes initiaux** : elles ne doivent pas être rejouées. Avant chaque nouvelle séance, réexécuter le script si vous souhaitez repartir de l'état initial.

Partie 1 — Programmation procédurale avec PL/pgSQL

Objectif

Utiliser des variables, des conditions, des boucles et créer une première fonction PL/pgSQL.

Rappel syntaxe / Exemple

Un bloc `DO` permet d'exécuter un traitement procédural sans créer de fonction permanente. La partie `DECLARE` contient les variables, `BEGIN ... END` les instructions, et `RAISE NOTICE` affiche un message.

```
DO $$
DECLARE
    montant NUMERIC = 12000;
BEGIN
    IF montant >= 10000 THEN
        RAISE NOTICE 'Montant important : %', montant;
    ELSE
        RAISE NOTICE 'Montant ordinaire';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Une fonction retourne un résultat et peut être appelée dans un `SELECT` :

```
CREATE OR REPLACE FUNCTION doubler(valeur NUMERIC)
RETURNS NUMERIC AS
$$
BEGIN
    RETURN valeur * 2;
END;
$$ LANGUAGE plpgsql;

SELECT doubler(250);
```

Pour parcourir des valeurs : `FOR i IN 1..5 LOOP ... END LOOP;`.

Exercice 1 — Variables et conditions

1. Écrire un bloc `DO` qui déclare un montant de retrait de 7 000 DA et affiche s'il dépasse ou non 10 000 DA.
2. Modifier le bloc pour distinguer trois catégories : moins de 5 000 DA, de 5 000 à 20 000 DA inclus, et plus de 20 000 DA.
3. Tester les montants 4 000, 12 000 et 25 000 DA. Vérifier les messages affichés.

Exercice 2 — Boucles

1. Écrire un bloc qui affiche les entiers de 1 à 5 à l'aide d'une boucle `FOR`.
2. Modifier le bloc pour calculer la somme des entiers de 1 à 10.
3. Afficher le résultat final à l'aide de `RAISE NOTICE`.

Exercice 3 — Première fonction métier

1. Créer une fonction `calculer_bonus(montant NUMERIC)` qui retourne 2 % du montant reçu.
2. Tester la fonction avec 10 000 et 50 000 DA.

Exercice 4 — fonction avancee

1. Créer une fonction `est_solde_suffisant(id INTEGER, retrait NUMERIC)` retournant `BOOLEAN` selon que le compte possède un solde suffisant pour le retrait demandé.
2. Tester la fonction sur les comptes 101 et 106 avec plusieurs montants. Justifier les résultats.

Remarque

Une fonction de consultation ne doit pas modifier les soldes. En cas de réinitialisation du dataset, les fonctions créées dans le schéma seront également supprimées.